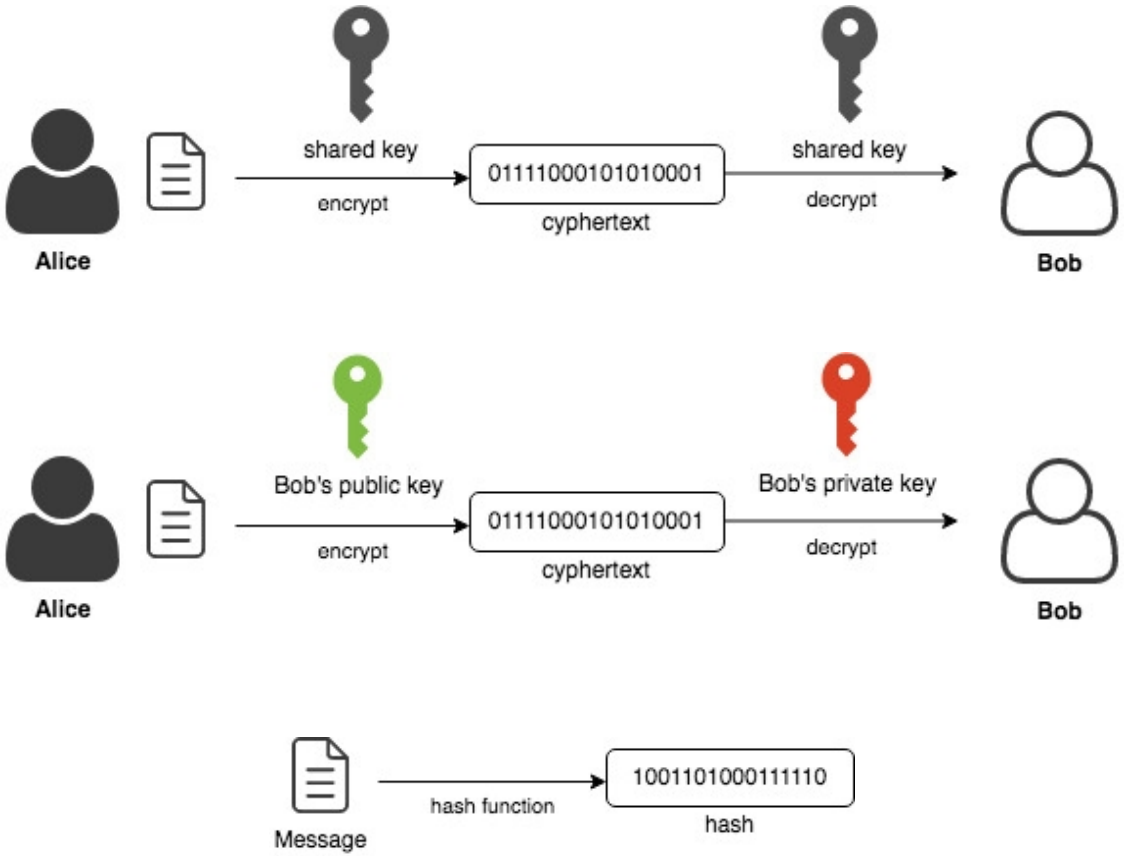
Authentication & Session Management

Secure Programming

Michael Lehmann

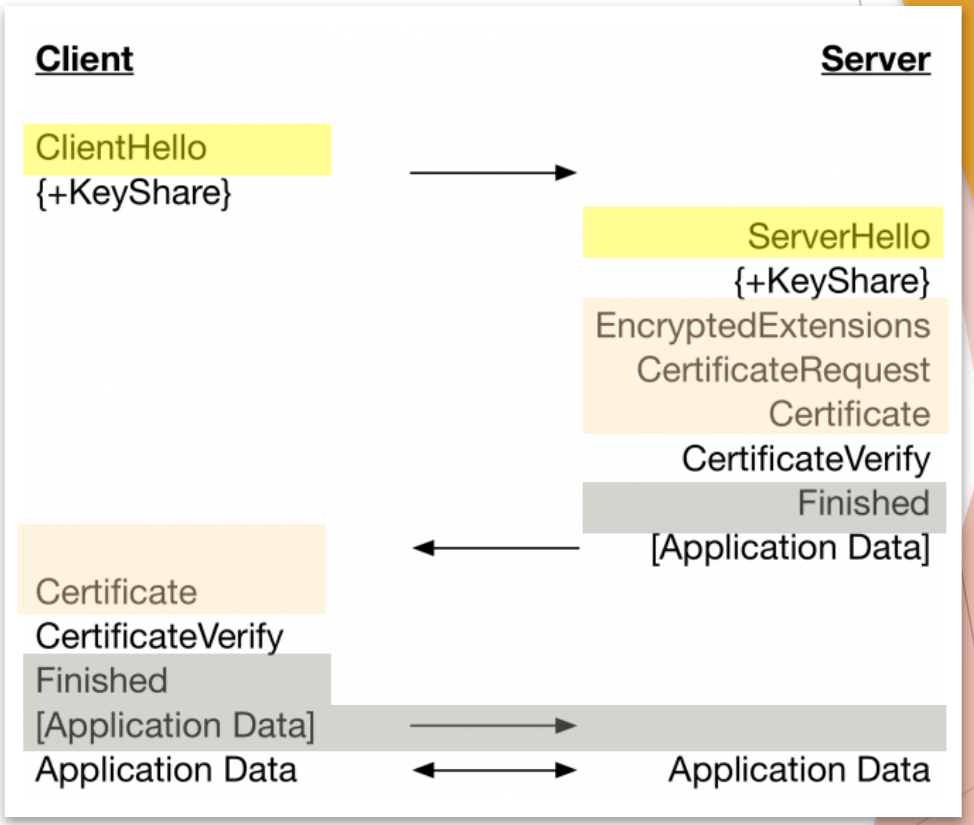
Basics

Cryptographic functions



TLS 1.3 overview

- Versions < 1.2 are vulnerable
- mTLS: client and server have a certificate
- Challenges in practice
  - Bad implementation of protocol or arithmetic (timing-attacks)
  - Misconfiguration
  - Weak and vulnerable CAs (browsers trust-list too big)
  - (Corrupt) Government



Authentication

Authorization

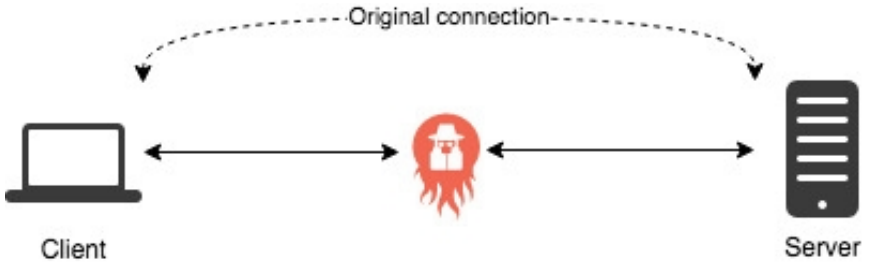
What you can do

Authentication

Who you are

Man-In-The-Middle attack (MITM)

- Man-in-the-middle pretends to be the server towards the client and vice-versa



- TLS prevents MITM (to a certain extend)

  → What circumstances could still lead to MITM?

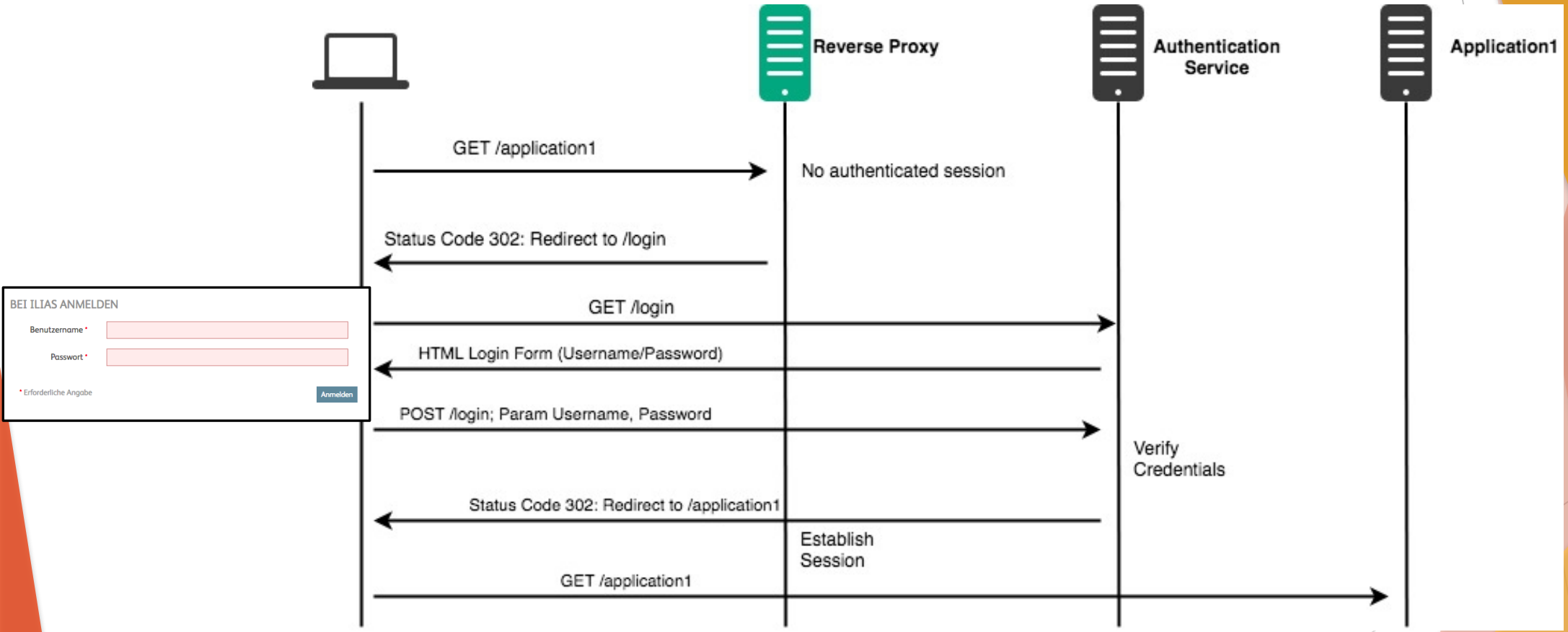
Authentication Terminology

Subject	Principal	Credential
• User identity	• Identifying information	• Proof of identity

Several mechanisms for authentication exist

- Username / Password
- Client Certificate
- Biometric
- Context-based

Authentication can be delegated



Not only implementation issues lead to broken authentication

- Automated attacks
- Social Engineering
- Poor design
- Implementation vulnerabilities

You need a set of measures for protection

- Unique username
- Password policy
- Correct password storage
- No default credentials
- Generic error messages
- Temporary lockout
- Limit extensive traffic
- https everywhere
- Extensive protection against attacks
- Strong authentication

Strong authentication requires multiple factors

Authentication Implementation Problem

Example pseudo-code for login:

```
IF USER_EXISTS(username) THEN
    password_hash=HASH(password)
    IS_VALID=LOOKUP_CREDENTIALS_IN_STORE(username, password_hash)
    IF NOT IS_VALID THEN
        RETURN Error("Invalid Username or Password!")
    ENDIF
ELSE
   RETURN Error("Invalid Username or Password!")
ENDIF
```

→ What is wrong here?

Session Management

There are mainly two attacks on sessions

Session Hijacking

- Stealing a session id from a victim’s system

Session Fixation

- “Planting” a session id into a victim’s system

«Planting» a session in the victim’s browser

1. Establish anonymous session
   - https://application.ch/login?sessionid=abcde
2. Make the victim click on the fabricated link
3. Victim logs in
4. Attacker can use the planted session id

→ What can you do about it? How can you protect from this problem?

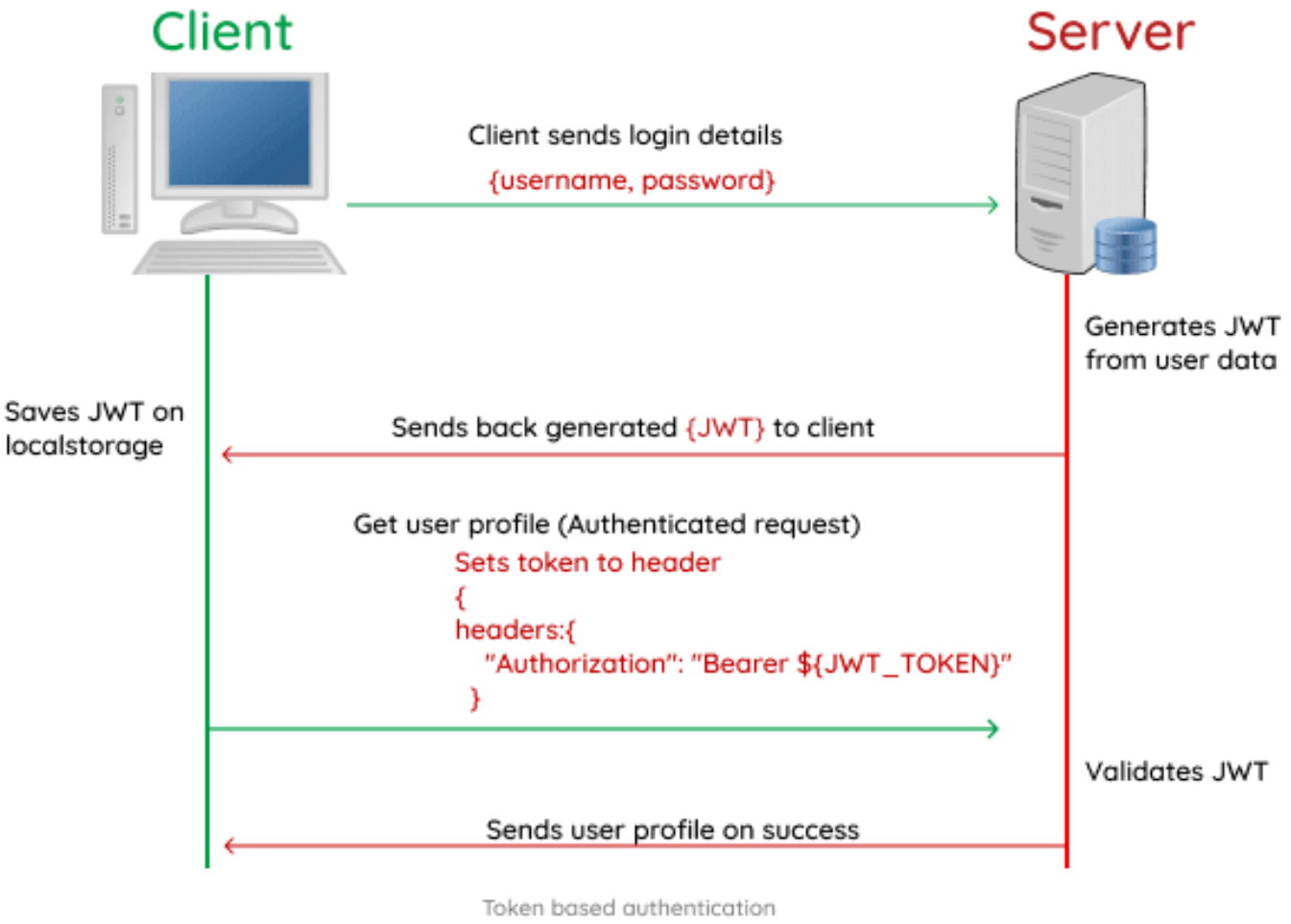
Handle the session id with care

- Long and random session id
- Generation on server-side on successful login
- Session expiration and proper invalidation
- Manual logout
- Invalidate all sessions on password change

Do not expose session id unnecessarily

- Set as cookie
- Protection of cookie with attributes
- Validation of cookies
- Binding to other user properties
- Usage of built-in session management

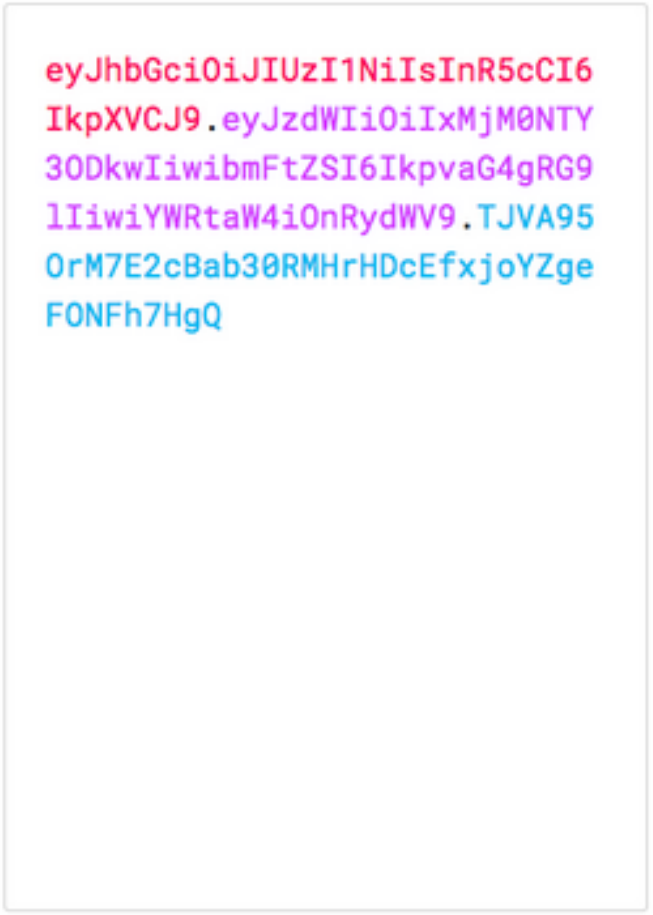
Tokens can be used instead of a session id



Source: dev.to

JWT consist of three parts

Encoded



Decoded

Handle tokens with care

- Protect sensitive information
- Ensure algorithm “None” is not accepted
- Implement logout
- Limit validity time
- Use a JWT library

More information: OWASP Cheat Sheet JWT

Excercises

- https://overthewire.org/wargames/natas/